



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

INGENIERÍA EN SOFTWARE

**Analisis y Diseño de Software
30724**

Presenta:

Altamirano Xavier

Ayo Alisson

Cordero Martin

Grupo: 1

Fecha:

21 de Mayo de 2026



 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

1. Objetivo

Analizar e implementar los patrones de diseño Abstract Factory, Singleton y Factory Method en el desarrollo de un CRUD de estudiantes, aplicando conceptos de análisis y diseño de software para mejorar la organización, reutilización, mantenimiento y escalabilidad de la aplicación.

2. Desarrollo

2.1. Patrones creacionales

2.1.1. Singleton

Descripción:

El Singleton es un patrón de diseño de tipo creacional cuyo objetivo principal es que una clase tenga una única instancia. Este patrón es especialmente útil cuando se necesita exactamente un objeto para coordinar acciones a través de un sistema, como en el caso de gestores de bases de datos, sistemas de registro como logging y el manejo de archivos.

Además de garantizar la unicidad, el Singleton proporciona un punto de acceso global a dicha instancia, permitiendo que sea alcanzable desde cualquier parte del programa sin el riesgo de que el objeto sea sobrescrito por otro código

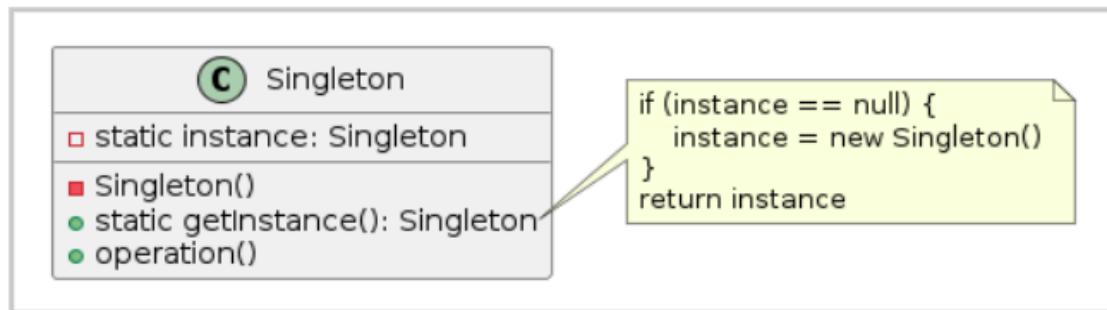
Características

- **Instancia Única:** Garantiza que no existan múltiples copias de la misma clase en memoria.
- **Constructor Privado:** Se oculta el constructor por defecto para evitar que otros objetos utilicen el operador new.
- **Método Estático de Creación:** Incluye un método (comúnmente llamado getInstance) que actúa como constructor especial: crea el objeto en la primera llamada y lo devuelve en las llamadas subsiguientes.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

- Inicialización Diferida (Lazy Initialization): El objeto solo se crea en el momento en que se solicita por primera vez, ahorrando recursos del sistema.
- Control de Acceso Global: Actúa de forma similar a una variable global, pero con una protección que impide que la instancia sea sustituida

Diagrama de clases:



 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Ventajas y desventajas:

Ventajas

- **Certeza de Instancia Única:** Se tiene el control total sobre el número de instancias disponibles.
- **Acceso Global:** Facilita el uso del objeto desde cualquier módulo del sistema sin pasar referencias constantemente.
- **Eficiencia de Memoria:** Al compartir una única instancia y usar inicialización diferida, se optimiza el uso de memoria comparado con variables globales que consumen recursos desde el inicio.
- **Base para otros patrones:** Sirve como fundamento para patrones como *Abstract Factory*, *Builder* o *Prototype*.

Desventajas:

- **Vulneración del SRP:** El patrón resuelve dos problemas a la vez (unicidad y acceso global), violando el **Principio de Responsabilidad Única**.
- **Acoplamiento y Mal Diseño:** Puede enmascarar un diseño deficiente donde los componentes dependen demasiado de un estado global oculto.
- **Dificultad en Pruebas Unitarias:** Muchos frameworks de prueba dependen de la herencia para crear objetos simulados (*mocks*). Dado que el Singleton tiene un constructor privado, es difícil de sustituir, lo que complica el testeo.
- **Problemas en Multihilo:** En entornos con múltiples hilos, requiere lógica adicional compleja para evitar que se creen varias instancias simultáneamente.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

2.1.2. Factory Method

Descripción:

Factory Method es un patrón de diseño de tipo creacional, este define una interfaz para la creación de objetos en una superclase, lo que permite que las subclases modifiquen simultáneamente el tipo de objetos que se crearán, en lugar de instanciar objetos mediante el operador new, este patrón propone la innovación de un método llamado “fábrica”.

Su objetivo principal es suavizar el acoplamiento entre una clase y los objetos que instancia. Este diseño funciona delegando la responsabilidad de la instanciación a las subclases, de tal manera que la base del creador puede trabajar con una interfaz producto genérica, mientras que las subclases deciden qué implementación se debe producir según el contexto.

Características

Sus principales características se centran en la flexibilidad y la adherencia a las buenas prácticas de ingeniería de software.

Interfaz Común para Productos: Todos los objetos devueltos por el método fábrica deben seguir una interfaz o clase base común, lo que permite que el código cliente trate a los productos de forma polimórfica.

Encapsulamiento de la Creación: Centraliza la lógica de construcción de objetos en un solo lugar, facilitando el mantenimiento y la evolución del sistema.

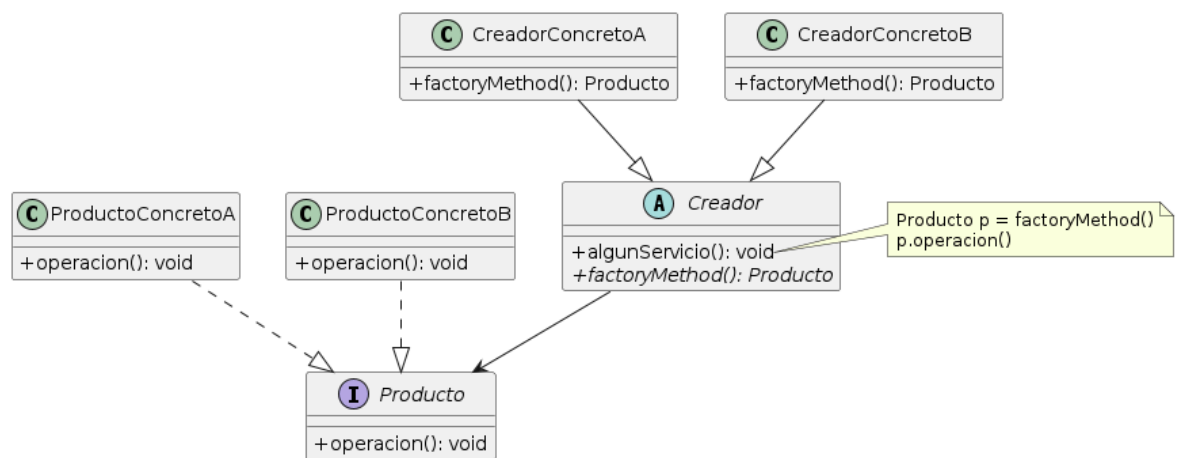
Principio de Responsabilidad Única (SRP): Permite mover el código de creación de productos a un lugar específico del programa, evitando que las clases de lógica de negocio se saturen con detalles de instanciación.

	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Principio de Abierto/Cerrado (OCP): Es posible introducir nuevos tipos de productos en la aplicación sin necesidad de modificar el código cliente existente, simplemente creando nuevas subclases creadoras.

Desacoplamiento: El código que utiliza el producto (código cliente) no conoce las clases concretas que se instancian, sino que interactúa únicamente con la interfaz abstracta.

Diagrama de clases:



Ventajas y desventajas:

Ventajas

- Evita el acoplamiento fuerte entre el creador y los productos concretos.
- Facilita la extensión del código (Principio de Abierto/Cerrado), permitiendo añadir productos sin romper el sistema.
- Centraliza la creación de objetos, lo que simplifica el mantenimiento y la aplicación del Principio de Responsabilidad Única.
- Permite la reutilización de recursos, como conexiones de bases de datos o sistemas de archivos, al poder devolver objetos existentes de una caché en lugar de crear siempre nuevos.

	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Desventajas

- Aumento de la complejidad del código, ya que requiere la creación de múltiples subclases nuevas para cada tipo de producto.
- Dificultad de lectura inicial para desarrolladores no familiarizados con patrones, el flujo de creación puede parecer indirecto o innecesariamente complicado.

2.1.3. Abstract Factory

Descripción:

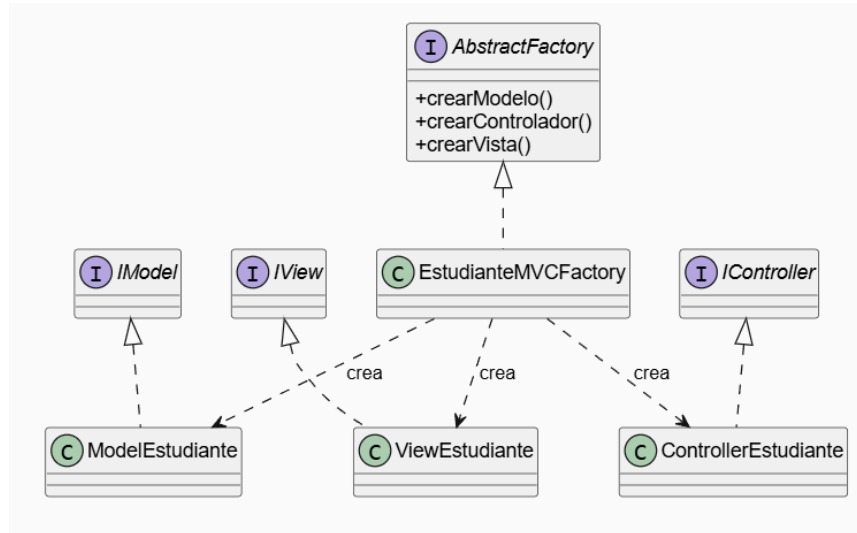
El patrón Abstract Factory proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin necesidad de especificar sus clases concretas. En lugar de crear objetos directamente usando la palabra clave new, el código cliente delega la creación a un objeto fábrica que se encarga de instanciar los productos adecuados.

Características:

- **Creación de familias de productos:** Se utiliza cuando un sistema debe ser configurado con una de múltiples familias de productos (por ejemplo, elementos de interfaz gráfica como botones y barras de desplazamiento que deben verse de manera consistente en un entorno Windows o Mac).
- **Abstracción:** Separa los detalles de implementación de las clases concretas (la fábrica concreta) de la interfaz de la que dependen los clientes (la fábrica abstracta).
- **Composición de objetos:** Favorece la composición sobre la herencia para la creación de los objetos, delegando la responsabilidad a una o más fábricas.

Diagrama de clases:

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin



En este CRUD de estudiantes, el patrón Abstract Factory se utiliza para crear de manera organizada los componentes principales del sistema MVC: el modelo, el controlador y la vista.

La clase **EstudianteMVCFactory** actúa como una fábrica que genera todos los objetos necesarios para que el CRUD funcione correctamente. De esta manera, la aplicación no necesita crear directamente las clases concretas, sino que delega esa responsabilidad a la fábrica.

Por ejemplo:

- el modelo administra los datos de los estudiantes,
- el controlador procesa las operaciones CRUD,
- y la vista muestra la interfaz gráfica al usuario.

Este patrón ayuda a mantener el sistema más ordenado, flexible y fácil de mantener, ya que separa la lógica de creación de objetos del funcionamiento principal de la aplicación.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Ventajas y desventajas:

Ventajas	Desventajas
Aislamiento de clases concretas: El cliente interactúa únicamente con interfaces abstractas y no necesita conocer los detalles de las clases concretas que se instancian, reduciendo el acoplamiento.	Dificultad para añadir nuevos tipos de productos: Si necesitas añadir un nuevo tipo de producto a la familia, deberás modificar la interfaz de la fábrica abstracta, lo que implica tener que modificar todas las clases de fábricas concretas existentes, rompiendo el principio de Abierto/Cerrado (Open/Closed Principle).
Intercambio fácil de familias de productos: Cambiar la familia entera de productos que utiliza un sistema es tan fácil como cambiar la instancia de la fábrica concreta en un solo lugar del código.	Complejidad adicional: Aumenta considerablemente el número de interfaces y clases en el sistema (patrón de muchas clases abstractas y concretas), lo que puede complicar el código fuente más de lo necesario en proyectos pequeños o sencillos.
Consistencia garantizada: Asegura que los productos generados por una fábrica concreta estén diseñados para trabajar juntos, evitando que el cliente mezcle objetos incompatibles.	

3. **Conclusión técnica**

4. **Recomendaciones**